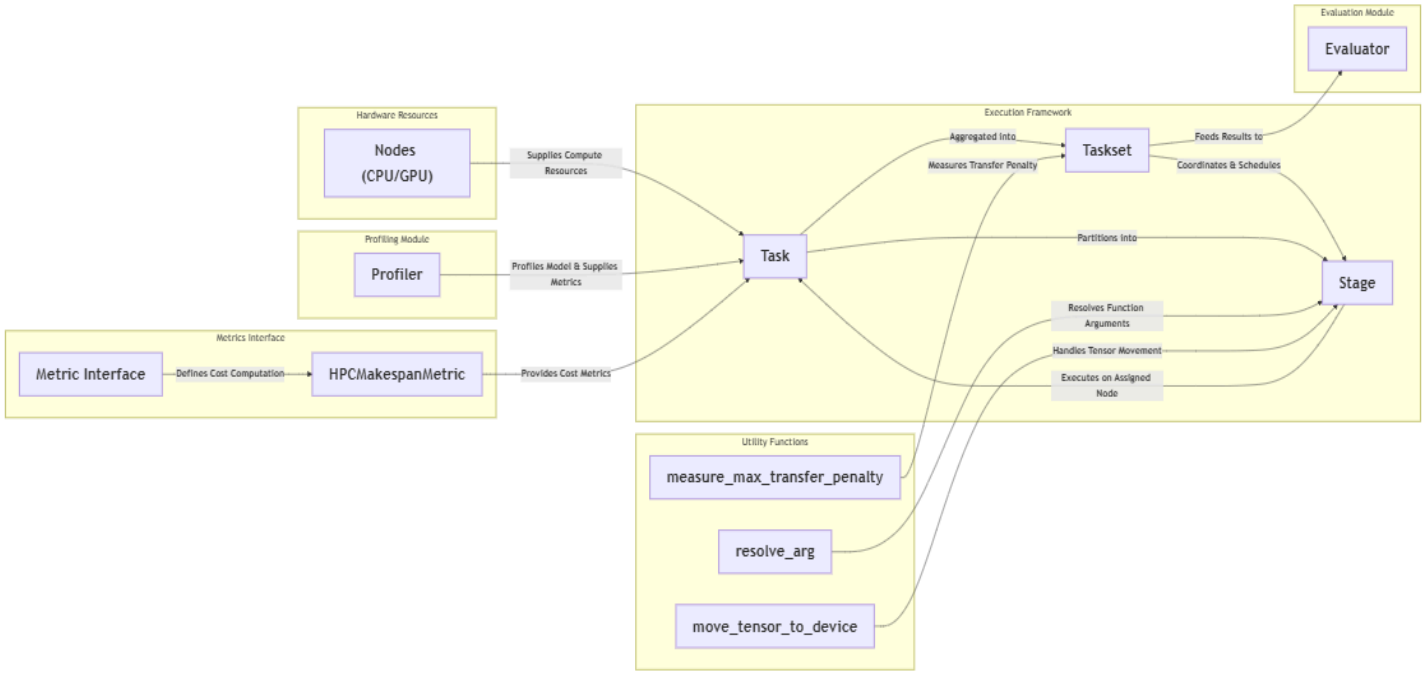
Project Report and Description

PyDart

Capstone Research Project:
A lightweight library to accelerate DNN inference tasks on heterogeneous compute.

Quick Links:https://github.com/parthshinde1221/pyDART/tree/research



1. Introduction and Motivation

Initial Goal

- Develop a scheduling solution to accelerate DNN inference on heterogeneous hardware (CPU/GPU).
- Heavily inspired by DART and PipeEdge, focusing on dynamic programming (DP) to partition model layers and distribute workloads.

Adapted/Pivoted Scope

- Observed that the same scheduling and partitioning logic can apply to HPC (High-Performance Computing) and batch workloads, as well as real-time scenarios that may feature multiple models being processed with varying input rates
- Aimed to build PyDart as a flexible, modular framework that can handle or be modified to handle:
    - **Real-Time Systems:** With periodic or sporadic task arrivals.
    - **Batch/HPC Environments:** Large-scale task batching in data centers.
    - **Edge Environments:** Resource-constrained but parallel-capable devices.

****Note** - While PyDart currently omits explicit deadline handling, an HPC or batch scenario—where high priority tasks commonly start together at t=0 and have a common deadline—can be viewed as a simplified case of real-time scheduling. The same dynamic programming and DAG-partitioning principles can be extended to incorporate more complex arrival patterns and deadlines if needed.

2. Key Insights and Approach

Core DP-Based Partitioning

- A **list-partition DP algorithm** constitutes the central logic, operating on a static DAG (Directed Acyclic Graph) extracted from the model.
- Designed to **minimize makespan** (time at which all tasks finish), but can be extended to other custom metrics (e.g., utilization, GPU time).

Lazy Execution Framework

- The library's workflow is divided into **three key phases**:
    1. **Initialization:** Node discovery, model profiling, DAG construction, DP-based partitioning.
    2. **Evaluation:** Running experiments (heavy vs. light tasks, naive vs. parallel) to determine optimal configurations.
    3. **Runtime:** Executing tasks in a batch mode with the chosen optimal setup.

Profiling and DAG Representation

- **torch.fx for Graph Extraction:**
    - Converts a model's forward pass into a DAG of operator nodes, capturing placeholders, outputs, and intermediate ops.
- **Detailed Timing and Caching:**
    - Collects per-node execution times across available hardware.
    - Caches common (model, node) pairs to avoid redundant profiling.

3. Iterative Development, Other Methods, and Learning.

Transition from Layer-Based to DAG-Based Profiling

- Initially used a **layer-by-layer profiling approach**, but this failed to capture **intermediate operations** effectively.
- Shifted to a **DAG-based method using `torch.fx`**, ensuring that:
    - **Every operation, including intermediate tensor manipulations, is profiled.**
    - **Partitioning decisions are based on a more holistic model execution graph rather than independent layers.**

Model Cloning and Profiling Optimization

- **Issue:** Model cloning was **initially required for instrumentation** during profiling, enabling **reuse of profiling data for similar tasks across multiple nodes**.
- **Fixes & Refinements:**
    - **Deepcopy vs. Reinitialization:**
        - **Deepcopying alone** led to **unexpected issues** for models with **lazy initialization or shared internal buffers**.
        - **We adopted a hybrid approach**, using **deepcopying where possible and selective re-initialization where needed** to maintain consistency.
    - **Profiling Cache System:**
        - Avoids redundant profiling by **storing execution times of previously seen models and input shapes.**
        - **Resolved excessive re-profiling at both task and experiment levels.**
    - **Profiler DAG Adjustments:**
        - **Hooks were attached to every node**, ensuring complete function tracing.
        - **Zero-based nodes (placeholders, outputs) were handled separately** to prevent profiling distortions.

Scheduling Approaches

Round Robin / Simple Async Scheduling (Discarded)

- Implemented **RR-based scheduling** but observed **no speedup**.
- **Slower than naive sequential execution in most cases** due to:
    - **Inefficient task transitions.**

    - **Overhead from frequent context switching between nodes.**
    - **Load imbalance when some tasks required significantly more compute time than others.**

Dynamic Programming (DP) Partitioning (Implemented & Optimized)

- **Achieved superior load balancing** by **globally optimizing how the DAG was split across nodes.**
- DP ensures that **each compute node receives an optimal number of layers**, leading to:
    - **More parallel execution.**
    - **Reduced idle time per node.**
    - **Minimized transfer and synchronization overheads.**

Profiler Improvements

- **Issue:** Stack assertion failures during re-profiling.
- **Fixes & Refinements:**
    - **Recompiling the DAG after attaching hooks** ensured that profiling did not interfere with model execution.
    - **Switched from complete aggregation to `key_averages` profiling** (without artificial limits).
    - **Zero-based nodes (placeholders, outputs) were explicitly handled**, preventing erroneous execution flow.
    - **Introduced a profiling cache to prevent redundant re-profiling for each task.**

Stage Execution Refinements

- **Issue:** Transfer time inconsistencies and incorrect argument resolution across stages.
- **Fixes & Refinements:**
    - **Explicit tensor movement handling between nodes** to prevent data inconsistency issues.
    - **Improved function argument passing between stages**, ensuring correct input propagation.
    - **Differentiated between transfer time and busy time,** allowing for more accurate performance analysis.
    - **Introduced custom operator handling**, improving flexibility for specialized operations.

Experiment Execution Optimization

- **Issue:** Stack assertion failures due to repeated profiling.
- **Fixes & Refinements:**
    - **Profiling is now done at the experiment level instead of per-task level.**
    - **Each experiment is isolated by removing the previously profiled CSV files**, ensuring a clean state for profiling runs.
    - **Prevented redundant re-profiling across independent experiment runs.**

Prototype with C++/pybind11 (Explored & Discarded)

- **Considered rewriting task-handling logic in C++** to reduce Python's **Global Interpreter Lock (GIL) bottleneck**.
- Ultimately **discarded** because:
    - **PyTorch's C++ backend already releases the GIL** during tensor operations.
    - **Added complexity was unnecessary** for the observed performance improvements.

4. Implementation Outline

Initialization Phase

1. **Node Discovery and Worker Threads:**
    - Identifies CPUs/GPUs.
    - Each node spawns a daemon worker thread with a FIFO queue for stage execution.
2. **Task Creation and Model Setup:**
    - User wraps each model and input data into a Task object.
    - A Profiler is attached to gather operator-level timing data.
3. **Profiling and DAG Construction:**
    - Models are traced via torch.fx, producing a DAG.
    - The Profiler runs each model on each node to collect per-node timings.
    - Results are stored in a DataFrame or CSV, enabling caching.
4. **Dynamic Programming Partitioning:**
    - The DAG is topologically sorted.
    - A DP algorithm divides the node list into stages, assigning them to nodes in a way that minimizes the maximum load (makespan).

Evaluation Phase

1. **Experiment Setup:**
    - The user tests various mixes of heavy vs. light models.
    - Baseline comparisons include naive sequential, simple async, or round-robin.
2. **Taskset Execution via `execute_all`:**
    - For each task, a separate thread is created.
    - **Concurrent Stage Assignment:**
        - Each task **concurrently** assigns its stages to the worker nodes (i.e., each stage is enqueued via `assign_task` on the appropriate node's queue).
    - **Pipelined Execution on Worker Threads:**
        - On each node, the worker thread processes these stages in a **predetermined (topological) order** for that task, but since there are multiple tasks, the overall system parallelism emerges.
        - This pipelined format—multiple worker threads executing stages from different tasks concurrently—is what drives global parallelism and accelerated throughput.

3. **Metric Collection and Comparison:**
    - Gathers makespan, throughput, speedup, and checks output correctness.
    - Identifies the best ratio or scheduling arrangement.

Runtime Phase

1. **Applying Optimal Configuration:**
    - Based on evaluation, the user sets the desired heavy/light ratio or partitioning scheme.
    - Large-scale Tasksets (batches) are formed following this ratio.
2. **Batch Execution:**
    - Calls `execute_all` to run tasks in parallel.
    - Each node's worker thread enforces correct stage order within each task while still handling tasks from multiple tasks in a concurrent, pipelined manner.
3. **Monitoring and Adaptation:**
    - Observes final metrics (e.g., overall throughput).
    - If workloads change (e.g., new models, different batch sizes), a re-evaluation or partial re-profiling can be done.

Results and Observations

1. Speedup Trends

- **Gradual Increase Beyond ~0.3 Heavy Ratio**
  - At least 30% heavy tasks are needed for noticeable performance gains.
  - Speedup remains modest below this threshold.
- **Peak at 0.7–0.9 Heavy Ratio**
  - Maximum speedups (often **1.8× [80% increase] on average, 2–3× [200-300%] in heavier-task scenarios**) are consistently observed in this range.
  - Confirms that high concurrency with heavier workloads yields the greatest acceleration.
- **Variability in Multi-GPU Systems**
  - More pronounced fluctuations on 10+ core and multi-GPU setups (e.g., A5000 setups), likely due to overheads in inter-device communication or task synchronization at extreme loads.(Though overall trends remain same.)

2. Throughput Trends

- **Consistent Outperformance Over Naive**
  - Parallel execution's throughput significantly exceeds naive methods across all experiments and hardware setups.
- **Stable Rise with Heavy Ratio**
  - Throughput climbs steadily until ~0.7–0.8 heavy tasks; slight plateau or saturation can appear beyond 0.9, as GPU resources become fully utilized.
- **Influence of Hardware Scale**
  - Large multi-GPU servers see sharper throughput improvements (and steeper naive declines), reinforcing the benefit of PyDart's DP-based scheduling in high-load environments.

3. Makespan Reduction(Similar to Speedup)

- **Significant Decrease vs. Naive**
  - On average, the system shows a **1.8× speedup (~80% faster)** than naive sequential.
  - In heavier-task settings, makespan reductions can yield **2–3× (200–300% faster)** improvements.
- **Consistent Decline with Higher Heavy Ratio**
  - Confirms that assigning more computationally intensive layers to different nodes reduces total end-to-end execution time.
- **Minor Fluctuations at High Cores**
  - Observed small “jitters” in multi-GPU setups (e.g., A5000) at heavy ratios near 0.8–0.9, probably from scheduling overhead or memory transfer bottlenecks.

4. Scaling Across Different Hardware

- **Range of GPU/Cores**
  - Tested on different systems ,such as small 2-core CPU + T4 GPU , medium 8-core systems (e.g., 3050 GPU), and large servers with multiple high-end GPUs (e.g., two A5000s) with 8-10 cores.
- **Increases in Complexity**
  - More cores or GPUs add synchronization and data-transfer costs, but the library scales effectively.
  - Rare, intermittent output mismatches (occurring once or twice on a 32-core system) were mitigated by re-running or adjusting concurrency settings.
- **Minimum Task Count for Clear Gains**
  - At least **2×(number_of_cores)** tasks are needed to see strong parallel benefits, ensuring enough workload to exploit concurrency.

5. Additional Observations

- **ThreadPool vs. DP-Based Approach**
  - A naive async approach that spawns one thread per task is less efficient, suffering from overhead and resource contention.
- **Round-Robin (RR) and Constant Grouping**
  - Simple strategies do not match PyDart's DP partitioning in makespan or throughput.
- **Trends in Speedup and Throughput Graphs**
  - Show a clear rise correlating with the fraction of heavy tasks; occasional anomalies appear but do not negate the overall upward trend.
- **Testing Thoroughness**
  - Multiple scenarios and repeated runs confirm consistent patterns: speedup emerges strongly at ~0.3 heavy ratio, peaks near 0.7–0.9.

Adaptability: Real-Time, HPC, and Batch

1. Batch / HPC Environments

1. **Optimized Execution Time and Throughput**
    - **Current Fit:** PyDart's dynamic programming–based partitioning already excels at distributing heavy layers across multiple CPUs/GPUs.
    - **Result:** Makespan (total execution time) is significantly reduced, and throughput (tasks per second) increases—ideal for data centers or HPC clusters.
2. **Flexible Metrics for Load Balancing**
    - **Possible Tweaks:** If metrics like GPU memory usage, power constraints, or advanced HPC-specific considerations (e.g., multi-node scheduling) are important, these can be integrated into the DP cost function.
    - **Benefit:** PyDart's design allows you to replace or supplement "makespan" with custom metrics to better reflect HPC performance objectives (e.g., energy efficiency or specific resource constraints).
3. **Large-Scale Batching**
    - **Use Case:** Data centers typically receive many concurrent inference requests (large volumes of tasks arriving together at t=0 or in short bursts).
    - **PyDart's Advantage:** The library's "execute_all" method naturally handles batched tasks, and each node's worker thread keeps multiple tasks in flight to maximize resource utilization.

2. Real-Time Systems

1. **Foundation for Periodic / Sporadic Tasks**
    - **Current Fit:** While deadlines aren't explicitly enforced, the core partitioning logic (DAG-based) and concurrency management (worker threads) can handle multiple tasks arriving in different patterns.
    - **Benefit:** With modest extensions, PyDart can accommodate real-time arrivals, giving each task a specific slice of CPU/GPU time.
2. **Deadline-Aware Scheduling**
    - **Key Modification:** Real-time systems typically require advanced scheduling (e.g., EDF—Earliest Deadline First, priority queues, or DM-based scheduling) to ensure tasks are completed before their deadlines.

    - **Integration:** PyDart's worker-queue structure can be adapted to handle priority-based dispatch. The DP partitioning remains valid, but scheduling within each node's queue would shift from FCFS to a policy favoring urgent tasks.
3. **Periodic Model Inference**
    - **Use Case:** Self-driving cars or robotics often run multiple neural nets repeatedly (e.g., for sensor processing).
    - **PyDart's Adaptation:** Each new "wave" of arrivals can reuse the DAG partitioning and load metric computations. If deadlines or sporadic bursts matter, partial or incremental scheduling logic can be introduced to handle newly arrived tasks mid-execution.

3. Edge Clusters / Resource-Constrained Devices

1. **Partial DAG Partitioning for Offloading**
    - **Current Fit:** The library's DP approach can isolate compute-heavy layers and schedule them on more capable devices (e.g., a local GPU).
    - **Advantage:** On an edge device (with limited resources), PyDart identifies which layers can be offloaded for best performance, while the remainder runs locally.
2. **Incorporating Network / Transfer Costs**
    - **Key Modification:** Data transfer overhead might become a dominant factor if layers are offloaded to remote or semi-remote GPU nodes.
    - **Possible Enhancement:** Extend the load metric to include estimated transfer time or network constraints (bandwidth, latency), ensuring the partitioning algorithm places heavier layers appropriately.
3. **Balancing Small Cores with Bursty Inference**
    - **Use Case:** Edge clusters might have multiple low-power CPUs and a lightweight GPU. Tasks can arrive intermittently (e.g., from IoT devices).
    - **PyDart's Adaptation:** Even with minimal hardware, the library's concurrency model ensures parallel scheduling where possible. Incorporating priority or real-time policies can help if tasks arrive unexpectedly or in bursts.

7. Known Issues & Future work.

- **Stack Assertions / Occasional Output Mismatches:**
    - Mostly mitigated, but can still appear in very high concurrency or corner cases.
- **Vision Transformer Issues:**
    - Complex attention blocks or newer architectures/operators can tax the tracer, requiring more refined instrumentation,also need to investigate more about this. Additional investigation is needed for thorough support
- **Extended Real-Time Scheduling:**
    - Adding strict deadlines or advanced queue policies (EDF, DM) would further align PyDart with classical real-time scheduling theory.
- **Advanced Profiling Metrics:**
    - Potential to integrate more granular GPU metrics (e.g., memory bandwidth usage) for a richer DP cost function.

8. Conclusions

- **Scalable and Flexible:**
    - A robust approach to parallelizing DNN inference across heterogeneous compute nodes.
- **Significant Performance Gains:**
    - Demonstrated speedup and throughput improvements over naive or simple scheduling strategies.
- **Broad Applicability:**
    - Real-time tasks with multiple models, batch/HPC workloads, and smaller edge scenarios can benefit.
- **Future Directions:**
    - Enhanced real-time integration, extended HPC usage (multi-node clusters), and deeper profiling options.

9. Novelty and Differentiators

1. **Derived from DART & PipeEdge:**
    - This library explicitly takes cues from DART (originally in Caffe) and PipeEdge (focused on distributed inference).
    - **New in PyTorch:**
        - PyDart is fully implemented in Python/PyTorch. In contrast, DART used Caffe, which limits the range of supported architectures compared to torch.fx.
    - **On-Prem vs. Distributed:**
        - PipeEdge tackled distributed inference. PyDart focuses on on-prem parallelism (though it could be extended to distributed scenarios if needed).
2. **Extensibility for New Load Metrics:**
    - While the core DP algorithm is the same (list-partition), users can define **custom load metrics** to match their system requirements.
    - This significantly increases adaptability (e.g., focus on memory constraints, utilization, or power consumption).
3. **All-Python Integration with PyTorch:**
    - Easy to integrate into existing PyTorch workflows, benefiting from PyTorch's broad ecosystem.
    - Avoids rewriting critical ops in C++, relying on PyTorch's internal C++ runtime and GIL-release where needed.

10. Additional Documentation and Notes

- A **software engineering reference**—complete with system design and
- This report aims to highlight both the developmental reasoning and the core technical contributions, showcasing how PyDart's DAG-based DP partitioning can be leveraged for various scenarios.